Running Loops and Conditional Commands

Bash control structures: for, while, until, if / elif / else

```
for HOST in host1 host2; do echo $HOST; done
```

What You Will Learn

Loops

Repeat a command with for, while, until

Exit codes

Know if a command succeeded (0) or failed

Tests

Compare numbers, strings and files

Decisions

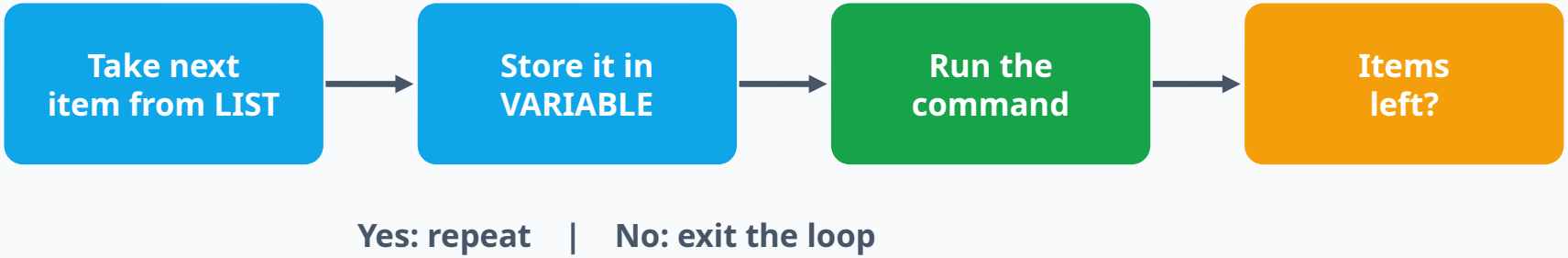
Run different tasks with if / elif / else

The for Loop

Repeat a task once per item in a list

```
for VARIABLE in LIST; do
    COMMAND $VARIABLE
done
```

Use it to check many targets: back up many databases, ping many hosts.



Where Does the LIST Come From?

Typed list

Command

```
for H in host1 host2 host3
```

Items processed

```
host1 host2 host3
```

Brace expansion

Command

```
for H in host{1..3}
for F in file{a..c}
```

Items processed

```
host1 host2 host3
filea fileb filec
```

Command output

Command

```
for N in $(seq 2 2 10)
```

Items processed

```
2 4 6 8 10
```

for Loop in Action

Print even numbers from 1 to 10

```
for EVEN in $(seq 2 2 10)
do echo "$EVEN"
done
```

seq FIRST STEP LAST makes the list.
Each number goes into $EVEN, one at a time.

Output

```
2
4
6
8
10
```

Tip: the variable name is your choice.

Exit Codes

Every command reports how it finished (0 to 255)

0
Success

1 - 255
Error

Read the last exit code with

```
echo $?
```

Leave a script early with exit N (no N = last command's code)

Exit Codes: true and false

```
$ /bin/true
$ echo $?
0
```

Always succeeds

```
$ /bin/false
$ echo $?
1
```

Always fails

Script example: #!/usr/bin/bash -> echo "Hello, world" -> exit 0

Testing a Condition

True = exit code 0. False = non-zero.

test command

```
test 1 -gt 0 ; echo $?
0
test 0 -gt 1 ; echo $?
1
```

[[...]] syntax

```
[[ 1 -eq 1 ]]; echo $?
0
[[ 1 -ne 1 ]]; echo $?
1
```

Spaces inside the brackets are mandatory: [[$x -gt 1]]

Numeric Operators

Operator	Meaning	Example	Result
-eq	equal	[[1 -eq 1]]	0 (true)
-ne	not equal	[[1 -ne 1]]	1 (false)
-gt	greater than	[[8 -gt 2]]	0 (true)
-ge	greater or equal	[[2 -ge 2]]	0 (true)
-lt	less than	[[2 -lt 2]]	1 (false)
-le	less or equal	[[3 -le 5]]	0 (true)

String and File Operators

String	Meaning
[[abc = abc]]	equal
[[abc != def]]	not equal
[[-z "$S"]]	string is empty
[[-n "$S"]]	string is not empty

File	Meaning
[[-f FILE]]	regular file exists
[[-d DIR]]	directory exists
[[-r FILE]]	you can read it

More operators: man test

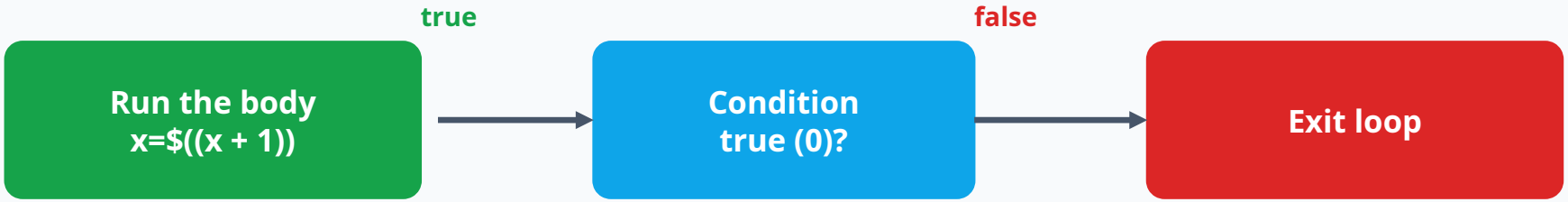
The while Loop

Repeat while the condition is TRUE

```
while <CONDITION>; do
    <STATEMENT>
done
```

```
x=1
while [[ $x -le 5 ]]; do
  echo "x is $x"
  x=$((x + 1))
done
```

```
x is 1
x is 2
x is 3
x is 4
x is 5
```



After the body runs, the condition is checked again

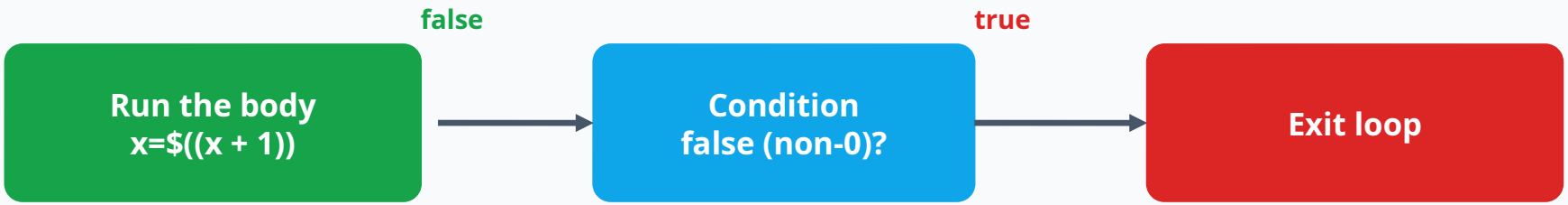
The until Loop

Repeat until the condition becomes TRUE

```
until <CONDITION>; do
    <STATEMENT>
done
```

```
x=5
until [[ $x -lt 1 ]]; do
  echo "x is $x"
  x=$((x - 1))
done
```

```
x is 5
x is 4
x is 3
x is 2
x is 1
```



After the body runs, the condition is checked again

while vs until

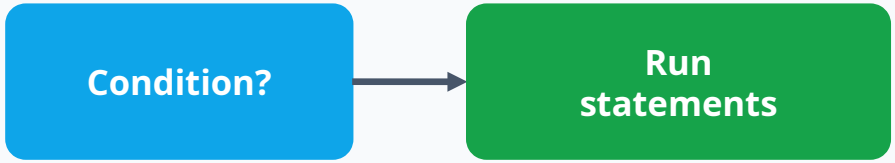
while	until
Runs while the condition is TRUE (exit code 0) Stops when it turns false	Runs while the condition is FALSE (non-zero) Stops when it turns true

Both test the condition BEFORE the first pass. If it blocks the first pass, the body never runs.

if / then

Do something only when a condition is true

```
if <CONDITION>; then
    <STATEMENT>
fi
```



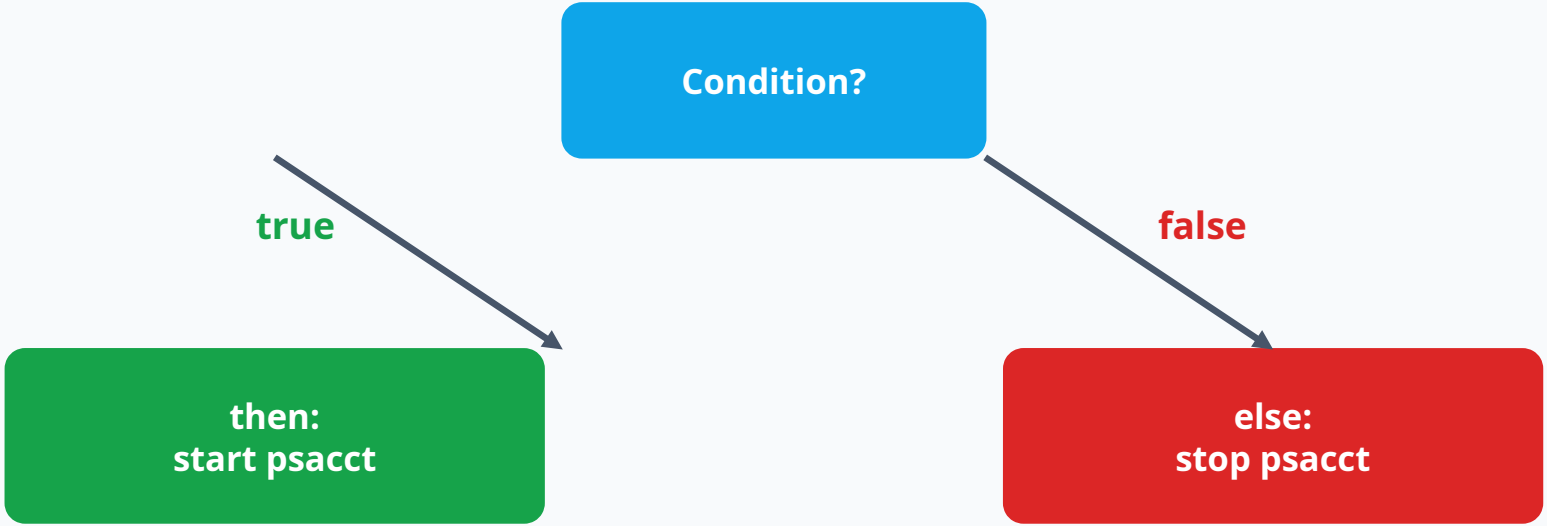
false: skip, nothing happens

Example: start psacct if it is not running

```
systemctl is-active psacct > /dev/null 2>&1
if [[ $? -ne 0 ]]; then sudo systemctl start psacct; fi
```

if / then / else

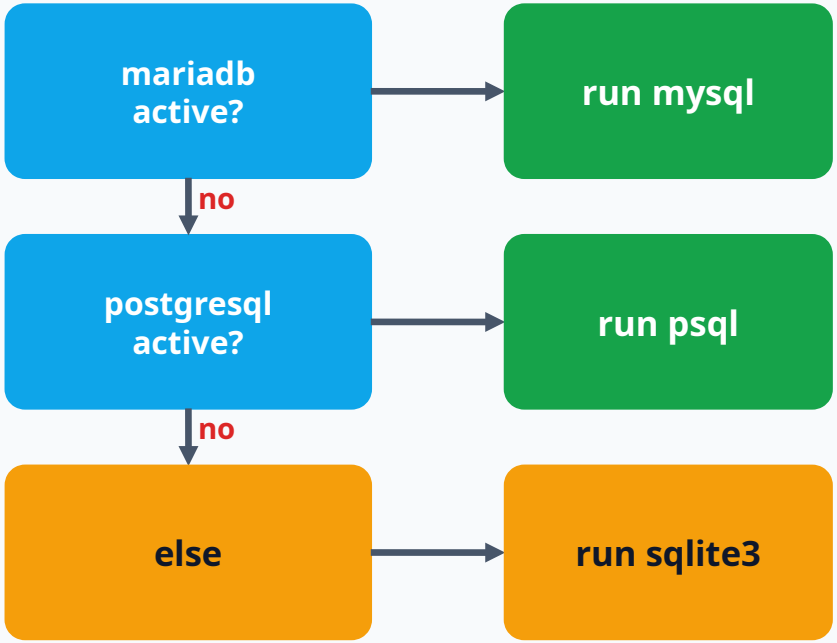
Pick one of two paths




```
if [[ $? -ne 0 ]]; then
    sudo systemctl start psacct
else
    sudo systemctl stop psacct
fi
```

if / elif / else

Test several conditions in order; first true wins




```
if [[ "$MARIADB_ACTIVE" -eq 0 ]]; then
    mysql
elif [[ "$POSTGRESQL_ACTIVE" -eq 0 ]]; then
    psql
else
    sqlite3
fi
```

Quick Reference

Construct	Use it when	Keywords
for	Run once per item in a list	for … in …; do … done
while	Repeat while a condition is true	while …; do … done
until	Repeat until a condition is true	until …; do … done
if / then	Act only if true	if …; then … fi
if / else	Choose between 2 paths	… else … fi
elif	Choose among many paths	… elif …; then …
$?	Check last exit code (0 = OK)	echo $?